



各位老师同学好，今天由我们组为大家做自然语言处理前沿相关的汇报，我们组选择的论文是1月份最火爆的大模型 deepseek r1，然后老师当时给我们的选题建议是这个偏通用，让我们补充一些论文和工作，于是我们就搜集了下deepseek r1之前的几篇重要的工作，并汇集了这份报告。

今天我们主要汇报的内容就是围绕着deepseek 24年12月的工作deepseek v3以及25年1月的工作deepseek r1，这里我们选择v3就是因为v3是deepseek前面一系列工作的集成者，而我们选择r1是因为这个模型在1月份那会特别出名，并且提出了新的强化学习的范式

这里v3最重要的特点即是其在使用了远小于其他开源模型的训练时长而达到了一个同样水平的模型，如左下角这个图，当时的llama3 405B使用了3080万H100GPU小时，而反观v3仅仅使用了279万H800GPU小时，在时间上仅仅有llama的不到十分之一，却能够达到一个与llama相当的效果，可以看出v3的时候就已经十分惊艳了

另外就是r1的工作，如这个柱状图，这里r1在好几个数据集上已经达到了超越当时最强模型o1，接下来我们就将介绍下deepseek这些工作里重要的几个工作

我们介绍的内容主要就是会分为这几个部分进行展开，首先就是v3系列的文章，我们将介绍v3以及其系列论文里的几个关键技术，包括moe mla mtp，以及就是r1的关键技术 grpo，以及最后会简单讲一下r1的训练过程

接下来是v3系列工作的介绍

首先我们介绍下moe，也就是混合专家模型，这里众所周知，大模型现在都是用的transformer的架构，而在transformer中占参数量大头的其实不是注意力机制，而是线性层，这里传统的线性层都会包括一个升维以及降维的操作，而这个操作过程中会有大型的矩阵运算，因为这里是一个稠密的升维降维矩阵，这里就存在问题，这里训练线性层的成本就是比较高的，因为非线性层每次运算都激活了所有参数，训练成本就非常高昂，于是这里就有工作提出可以把这里一个大的线性层拆成很多个小型网络，这里就把这些小型的网络称为专家，这里在输入的时候会有一个可

训练的路由网络，路由网络就是为了挑选出几个专家网络来进行工作，以此大大降低计算成本。通过这种方式，moe可以达到：相同参数下计算更小，相同计算下网络规模更大性能更好，

而传统的moe就存在几个问题，一个就是专家不够专精：现有 MoE 的专家数量有限，路由分配不够精细。导致分配给同一专家的 Token 涵盖了不同的知识类型（例如，既有代码 Token 也有金融 Token）。这里deepseek的优化方法就是致力于通过扩大专家池，并且在路由的时候选择更多的专家，提升专家的学习聚焦度

另一个就是知识冗余的问题，不同的专家都需要共同的基础知识（如基础语法、常识）。这导致多个专家会在各自的参数中重复存储和学习这些共享知识。旨在通过架构和训练算法创新，高效管理共享知识，鼓励专家将精力集中于学习独特的、非共享的知识，从而最大限度地提高每个参数的边际贡献

最后实验的效果也很不错，在同等的计算代价下，比稠密网络以及普通moe的情况有很大的提升，并且其性能做到了与每次都使用同等规模的稠密网络的性能相当，把moe的性能做到了极限

接下来我们介绍第二个技术，mla，也就是多头潜在注意力机制，这里我们先介绍下该技术的一个基础，也就是kv-cache

我们知道，一个transformer在进行推理时，要计算大量的q k v 矩阵，这里我们举一个具体的例子，我们现在输入了一个token，通过embedding以及q k v转换，我们成功计算了这里的注意力输出，这里接上一个分类头就是下一个token，然后我们就是把新生成的token拼到我们的输入里边，

这里我们第二张图，为了计算第二个token的下一个token，我们使用第二个token的Q向量分别和第一个token的K向量以及自己的K向量计算点积，通过softmax得到权重再乘上两个token的value向量，这里就可以得到第三个token，而这里第二个token计算的时候可以发现这里第一个token的k向量和v向量被反复的计算，于是这里很符合直觉的一个操作就是把之前计算过的k向量和v向量缓存下来，这样每次新的token进来，只需要计算新token的 q k v向量，再利用缓存的数据即可完成下一个token的计算，这样就可以节省大量的计算成本，通过缓存来加速计算，很明显这里就是通过空间来换时间的操作，而这里最大的问题就是，如果输入的文本越来越长，这里存入的k和v也就越来越多，对显存的要求也就越来越大

我们这里就介绍下mla的核心内容，其主要原理就是通过一个压缩矩阵和一个解压矩阵来缩小要缓存的内容大小，这里其实是类似一个lora的思想，通过把一个大矩阵，分解为两个小矩阵，就可以节省大量的计算开销

这里的改进思路类似lora，简单来讲就是将原来原来要存的一个大向量通过压缩矩阵和解压矩阵，把向量内容变小了

并且这里最后的实验还发现，在同等的模型参数量下，使用多头潜在注意力机制甚至是比原来使用多头注意力的效果还要好，并且这里在表里可以看出，使用MLA还达到了远远更低的kv cache，大幅度的减少了显存占用量

mtp这里就是把transformer block的最后一层进行了改进，把原来的一个头做预测改成了多个头做预测，原来的输入本来只是预测一个t2，这里就是同时预测t2 t3 t4，具体训练的时候再分配不同的权重

为什么这样做是合理的呢？因为人说话思考时，不是一个字一个字蹦出来的，我们的大脑通常是一下子规划出一个语义片段。MTP 正是模仿了这种人类的‘提前规划’能力。

这种多 Token 预测机制带来了两方面的巨大好处：

包括训练信号增强：MTP 约束模型在训练阶段就要一次性预测多个 Token，相当于给骨干网络增加了更多的监督信号，大大提高了数据利用率。以及特征规划：它迫使模型提前规划特征提取，让模型提取的特征必须能够更好地预测未来的多个 Token，从而增强了模型的长期一致性。”

虽然 MTP 的思想很好，但原版 MTP 存在一个致命缺陷：跨步预测的准确率太低，模型难以收敛。

随着预测的步数越远（比如预测第T+4个 Token），这个预测头是‘盲目’的，它无法获得前面已预测 Token 的信息（如T+1到T+3）。这种信息缺失极大地增加了预测难度，导致收敛非常不稳定

这里DeepSeek MTP 为每个 MTP 头设计了双源输入，解决了信息缺失。

这里的一个 MTP 头（比如预测 T+3 的头）的输入，不仅来自 DeepSeek 主网络的特征，更关

键的是，它还接收前一个 Token $T+2$ 的 Embedding 特征。通过将这些特征进行归一化、拼接和线性映射，每个 MTP 头都能在预测时具备更充分、更连贯的上下文信息。

最终效果：这种机制极大地提高了多步预测的准确性和训练的稳定性。最终的消融实验也证明，添加 DeepSeek MTP 训练后，无论是小模型还是大模型，性能都有显著提高。

接下来我们介绍 deepseek r1 的核心技术，也就是 GRPO

首先 GRPO 是一种强化学习，并且是一种基于策略的范式的方法，这里我们就简单介绍一下强化学习的相关概念

这里举一个游戏作为例子，State 就是当前的状态，而 agent 就是要训练的智能体，有一个策略函数，Agent 根据 State 来选择一个 Action，然后环境会给这个动作一个奖励

几个概念包括 Action Space，代表模型可选择的动作，Policy 策略函数，即输入 State 输出 Action，Trajectory 轨迹，代表一连串的状态和动作策略，Return 即游戏的汇报，代表从当前时间点 to 游戏结束的 reward 的累积和

我们目标即是训练一个 policy 神经网络 π ，在所有可能的 trajectory 中，得到的 return 的期望最大

这里具体的公式推导因为时间原因就不再详细说了，主要的点就在于把这里的优化的目标能够写成一个 loss 函数，然后使用梯度下降的方法来对这个函数进行优化

其中这里最后的形式就是要最大一个 $R(\tau) \times \log P_\theta(a|s)$ ，要让这个时最大的，这里直观理解就是，对于一个状态 s ，如果当前执行动作 a 的 reward 比较高，那就让模型更加可能的选择当前的动作 a

然而这里也存在另外的问题，就是对于不同的状态 s ，本身就存在好的状态以及差的状态，对于好的状态，不管执行什么样的策略都能拿到比较好的 reward，对于差的状态也是不论执行什么策略 reward 都差，这样好的状态会增加所有动作概率，差的状态减少所有动作概率，这样就不

是很合理，我们应该更希望让相对好的动作的概率增加，让相对差的动作概率减小，这里思路就是使用一种优势函数的方法，即对当前的状态本身会有价值的估计，然后使用不同的动作价值减去状态价值，得到一个优势的评估

这里计算优势函数的方法具体使用的广义优势估计，即对动作进行了多步的采样，这里我们就不介绍具体的公式了，简单来讲就是使用一种比较均衡的方法来计算得出优势函数，即当前动作价值比状态价值好多少

接下来就是到经典的PPO算法，这里原来的公式是一种on-policy的训练模型，而ppo为了提高样本效率，使用了一种重要性采样的方法，使用旧的策略生成样本来更新新策略从而提高效率。然而，为了防止新策略在数据复用过程中与旧策略偏离太远导致训练不稳定，PPO引入了截断机制。这个机制通过限制新旧策略的概率比率在一个很小的安全区间内，保证了每一次策略更新都是稳定且可靠的，实现了大步探索和安全学习的平衡。

从loss公式来看就是即使一个策略很好，其最大的参数更新也只有 $1+\epsilon$ 倍，反之如果一个策略很差，最大的更新也只有原来的 $1-\epsilon$ 倍

PPO的具体训练过程就是，先采样一组数据，然后结合奖励模型和状态价值模型来计算GAE广义优势估计

并且截断新旧策略的更新比例，限制更新幅度

这里PPO的缺点也很明显，过分复杂，要涉及好几个模型，包括一个训练的模型一个参考模型一个奖励模型和一个状态价值模型，其中需要更新的除了训练模型还有就是状态价值模型，这对于大模型的训练者有着极高的经验与技巧要求

而这里GRPO，最大的不同点也是比较核心创新的地方就是GRPO没有再使用一个状态价值模型来估计当前状态的价值，而是在首先的采样输出中，通过计算组内的奖励来得到一个相对优势，以此来进行策略更新

这里可以看一个GRPO训练的简单例子

模型此时不需要训练状态价值函数，就可以得到优势值，

具体方法就是先采样多个输出，然后让奖励模型对多个输出进行打分，优势函数的计算方法就是使用的标准化的方法，即减去均值除上方差，这样就可以直接去掉状态价值模型，而只需要训练单个模型

最后就是我们对deepseek r1训练阶段的简单介绍

接下来是我们对deepseek的训练过程的介绍，我们首先介绍下deepseek-r1-zero，这里这个比较有意思

DeepSeek-R1-Zero 颠覆了传统的预训练-监督微调-强化学习大模型训练范式。它最大的创新在于，直接在 DeepSeek-V3-Base 模型上跳过费时费力的 监督微调阶段，仅通过强化学习就训练出了具有强大推理能力的DeepSeek-R1-Zero。

这打破了模型训练的传统范式，其最直接的好处就是省去了 SFT 阶段所需的大量人工标注成本。不过R1-Zero 仍有局限性：它在遵循复杂指令和通用对话的稳健性方面仍有不足，存在着可读性差、语言混杂的问题。

R1-Zero主要的贡献是引发了业界对训练范式的思考：即仅使用强化学习是否能够成为一种足够好的训练范式？

另外，最终打败O1 的旗舰模型是DeepSeek-R1，接下来我们将介绍DeepSeek-R1的训练过程。

最终deepseek r1的训练过程总共包括四个阶段

第一步为SFT，使用数千条Long CoT数据来做冷启动，这里的CoT的数据来源就是之前的r1-zero生成的数据，目的是为了让RL的前期训练变得稳定，以及进一步增强推理能力

第二步则对SFT好的模型使用推理数据进行GRPO的强化学习得到了强化学习后的版本，此时的模型已经有了很好的推理能力，此时又通过这个模型来构建数据，通过拒绝采样的方法构建了60w条的高质量CoT推理数据

第三步则是根据上文的60w高质量推理数据以及v3基座模型中挑选20w条非推理数据，合并总共

80w条数据来对v3 base模型进行SFT，此时的模型已经有了很好的推理能力

第四步则是最后使用全场景的数据来进一步用GRPO来做与人类偏好的对齐，推升模型回复的有用性以及无害性，最终得到了deepseek r1

以上就是我们组汇报的全部内容，大家有什么问题或建议。